

54. Fetch

JS server ko **network requests** bhej sakta hai aur page reload kiye bina naya data la sakta hai (order submit, user info load, latest updates...). Isse **AJAX** kehte hain (Asynchronous JavaScript And XML, par XML zaruri nahi, naam purana hai).

`fetch()` modern aur versatile tarika hai.

```
js
let promise = fetch(url, [options]);
```

- `url`: kahan se
- `options`: `method`, `headers` waghera. Na do to simple **GET** request.

Response 2 stage me aata hai

Stage 1: server ke **headers** aate hi promise `Response` object ke saath resolve hota hai. Yaha status aur headers check kar sakte ho, body abhi nahi.

- Promise sirf tab **reject** hota hai jab request hi na ho paye (network problem, site hi nahi). **404, 500 jaise HTTP errors reject nahi karte.**
- `response.status`: HTTP code (jaise 200)
- `response.ok`: `true` agar status 200-299

```
js
let response = await fetch(url);
if (response.ok) {
  let json = await response.json();
} else {
  alert("HTTP-Error: " + response.status);
}
```

Stage 2: body ke liye alag method call:

- `response.text()`: text
 - `response.json()`: JSON parse
 - `response.formData()`: `FormData` object
 - `response.blob()`: `Blob`
 - `response.arrayBuffer()`: `ArrayBuffer`
 - `response.body`: `ReadableStream` (chunk-by-chunk)
-

```
js
```

```
let commits = await (await fetch(url)).json();
```

Ya promises se:

```
js
```

```
fetch(url).then(r => r.json()).then(commits => alert(commits[0].author.login));
```

Sirf ek body-reading method chun sakte ho. `text()` ke baad `json()` **fail** hoga (body pehle hi consume ho chuki).

Response headers

`response.headers` Map jaisa (poora Map nahi):

```
js
```

```
response.headers.get('Content-Type');  
for (let [key, value] of response.headers) { ... }
```

Request headers

```
js
```

```
fetch(url, { headers: { Authentication: 'secret' } });
```

Kuch headers **forbidden** hain (browser khud sambhalta hai): `Accept-Charset`, `Accept-Encoding`, `Connection`, `Content-Length`, `Cookie`, `Date`, `Host`, `Origin`, `Referer`, `Sec-*`, `Proxy-*` waghera.

POST requests

Options: `method`, `body`. `body` ho sakti hai:

- string (jaise JSON)
- `FormData` (`multipart/form-data`)
- `Blob`/`BufferSource` (binary)
- `URLSearchParams` (`x-www-form-urlencoded`), kam use)

JSON sabse zyaada:

```
js
```

```
let response = await fetch('/article/fetch/post/user', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json;charset=utf-8' },
  body: JSON.stringify(user)
});
let result = await response.json();
```

`body` string ho to `Content-Type` default `text/plain;charset=UTF-8` hota hai, isliye JSON ke liye header khud do.

Image bhejna

`Blob` ka apna `type` hota hai, jo automatically `Content-Type` ban jaata hai:

js

```
let blob = await new Promise(resolve => canvasElem.toBlob(resolve, 'image/png'))
let response = await fetch('/upload', { method: 'POST', body: blob });
```

Summary

Typical fetch = 2 `await`:

js

```
let response = await fetch(url, options); // headers aane par
let result = await response.json();      // body
```

- Response: `status`, `ok`, `headers`
- Body methods: `text`, `json`, `formData`, `blob`, `arrayBuffer`
- Options abhi tak: `method`, `headers`, `body`

Task: GitHub users fetch karna

`getUsers(names)`: har user ke liye ek `fetch`, requests ek doosre ka wait na karein, fail/404 par `null`.

js

```
async function getUsers(names) {  
  let jobs = names.map(name =>  
    fetch(`https://api.github.com/users/${name}`).then(  
      r => r.status !== 200 ? null : r.json(),  
      () => null  
    )  
  );  
  return await Promise.all(jobs);  
}
```

Yaha `.then` seedha `fetch` par lagaya taaki har response aate hi `.json()` padhna shuru kare, sab ka wait na kare.